

ObserverBase#

<https://pytorch.org/docs/stable/generated/torch.ao.quantization.observer.ObserverBase.html>

Description:

Base observer Module. Any observer implementation should derive from this class. Concrete observers should follow the same API. In forward, they will update the statistics of the observed Tensor. And they should provide a `calculate_qparams` function that computes the quantization parameters given the collected statistics. `dtype` – dtype argument to the quantize node needed to implement the reference model spec. `is_dynamic` (bool) – indicator for whether the observer is a placeholder for dynamic quantization quantization (or static) –

Function Signature

```
class torch.ao.quantization.observer.ObserverBase(dtype,  
is_dynamic=False)[source]#
```

Parameters

```
classmethod with_args(**kwargs)[source]#
```

```
classmethod with_callable_args(**kwargs)[source]#
```

Code Examples

```
>>> Foo.with_args = classmethod(_with_args)
>>> foo_builder = Foo.with_args(a=3, b=4).with_args(answer=42)
>>> foo_instance1 = foo_builder()
>>> foo_instance2 = foo_builder()
>>> id(foo_instance1) == id(foo_instance2)
False
```

```
>>> Foo.with_callable_args = classmethod(_with_callable_args)
>>> Foo.with_args = classmethod(_with_args)
>>> foo_builder =
Foo.with_callable_args(cur_time=get_time_func).with_args(name="d
an")
>>> foo_instance1 = foo_builder()
>>> # wait 50
>>> foo_instance2 = foo_builder()
>>> id(foo_instance1.creation_time) ==
id(foo_instance2.creation_time)
False
```

Detailed Documentation

Documentation

Base observer Module. Any observer implementation should derive from this class.

Concrete observers should follow the same API. In forward, they will update the statistics of the observed Tensor. And they should provide a `calculate_qparams` function that computes the quantization parameters given the collected statistics.

`dtype` – dtype argument to the quantize node needed to implement the reference model spec.

`is_dynamic` (bool) – indicator for whether the observer is a placeholder for dynamic quantization

`quantization` (or static) –

Wrapper that allows creation of class factories.

This can be useful when there is a need to create classes with the same constructor arguments, but different instances. Can be used in conjunction with `_callable_args`

Wrapper that allows creation of class factories args that need to be called at construction time.

This can be useful when there is a need to create classes with the same constructor arguments, but different instances and those arguments should only be calculated at construction time. Can be used in conjunction with `_with_args`

